

ISE 105 - Programlamaya Giriş

Fonksiyonlar

Dr. Öğr. Üyesi Muhammed Kotan

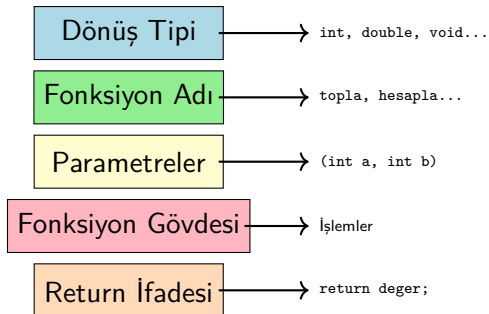
Bilişim Sistemleri Mühendisliği

2025 Güz



İçindekiler

- 1 Fonksiyonlara Giriş
- 2 Fonksiyon Tanımlama
- 3 Parametreler
- 4 Varsayılan Parametreler
- 5 Fonksiyon Overloading
- 6 Inline Fonksiyonlar
- 7 Statik Değişkenler
- 8 Özyinelemeli Fonksiyonlar
- 9 Uygulamalar
- 10 Alıştırmalar
- 11 Özet



Örnek:

```
int topla (int a, int b) {  
    int sonuc = a + b;  
    return sonuc;  
}
```

- Dönüş tipi → `int`
- Fonksiyon adı → `topla`
- Parametreler → `(int a, int b)`
- Fonksiyon gövdesi → `İşlemler`
- Return ifadesi → `return`

Basit Fonksiyon Örneği

Parametresiz:

```
#include <iostream>
using namespace std;

void selamla() {
    cout << "Merhaba!"
         << endl;
}

int main() {
    selamla();
    return 0;
}
```

Parametrelili:

```
#include <iostream>
using namespace std;

int topla(int a, int b) {
    return a + b;
}

int main() {
    int s = topla(5, 3);
    cout << s << endl;
    return 0;
}
```

Fonksiyon Prototipi

Neden Gerekli?

Fonksiyonun tanımından önce derleyiciye bildirim yapar.

```
#include <iostream>
using namespace std;

// Prototip (bildirim)
int carp(int x, int y);

int main() {
    cout << "Carpim: " << carp(4, 7) << endl;
    return 0;
}

// Tanım
int carp(int x, int y) {
    return x * y;
}
```

Değer ile Parametre Geçişi (Pass by Value)

```
#include <iostream>
using namespace std;

void degistir(int x) {
    x = 100;
    cout << "Fonksiyon icinde: "
         << x << endl;
}

int main() {
    int sayi = 50;
    degistir(sayi);
    cout << "Main icinde: "
         << sayi << endl;
    return 0;
}
```

Çıktı:

Fonksiyon icinde: 100

Main icinde: 50

Önemli! Kopya gönderilir, orijinal değişmez!

Referans ile Parametre Geçişi (Pass by Reference)

```
#include <iostream>
using namespace std;

void degistir(int &x) {
    x = 100;
    cout << "Fonksiyon icinde: "
         << x << endl;
}

int main() {
    int sayi = 50;
    degistir(sayi);
    cout << "Main icinde: "
         << sayi << endl;
    return 0;
}
```

Çıktı:

Fonksiyon icinde: 100

Main icinde: 100

Önemli!

& işareti ile orijinal değişken gönderilir ve değişir!

Const Referans Parametreler

Ne İşe Yarar?

Büyük veri yapılarını kopyalamadan göndermek, ancak değiştirmemek için kullanılır.

```
#include <iostream>
#include <string>
using namespace std;

void yazdir(const string &metin) {
    cout << metin << endl;
    // metin = "Yeni"; // HATA! const olduğu için değiştirilemez
}

int main() {
    string mesaj = "Merhaba Dünya";
    yazdir(mesaj); // Kopyalanmadan gönderilir, performans++
    return 0;
}
```


Tür	Sözdizimi	Değişir mi?
Değer	<code>int x</code>	Hayır
Referans	<code>int &x</code>	Evet
Const Referans	<code>const int &x</code>	Hayır

Genellikle

- Değiştirmek istiyorsanız: **Referans**
- Değiştirmeyeceksiniz ama büyük veri: **Const Referans**
- Küçük veri (`int`, `double`): **Değer**

Varsayılan Parametreler

```
#include <iostream>
using namespace std;

void selamla(string isim = "Misafir", int saat = 12) {
    cout << "Merhaba " << isim << "!" << endl;
    cout << "Saat: " << saat << endl;
}

int main() {
    selamla();                // Her ikisi de varsayılan
    selamla("Ali");           // Sadece isim verildi
    selamla("Ayse", 15);      // Her ikisi de verildi
    return 0;
}
```

Önemli Kural: Varsayılan parametreler sağdan başlamalıdır!

- **Doğru:** fonk(int a, int b = 5, int c = 10)
- **Yanlış:** fonk(int a = 1, int b, int c)

Fonksiyon Aşırı Yükleme (Overloading)

Tanım

Aynı isimde ama farklı parametre listelerine sahip birden fazla fonksiyon tanımlanabilir.

```
#include <iostream>
using namespace std;

int topla(int a, int b) {
    return a + b;
}

double topla(double a, double b) {
    return a + b;
}

int topla(int a, int b, int c) {
    return a + b + c;
}
```

Overloading Kuralları

Neye Göre Farklı Olabilir?

- Parametre sayısı farklı olabilir
- Parametre tipleri farklı olabilir
- Parametre sırası farklı olabilir

Yeterli Olmayan:

- Sadece dönüş tipi farklı olması yeterli değildir!
- Sadece parametre isimleri farklı olması yeterli değildir!

Örnek:

- **Geçerli:** `void fonk(int)` ve `void fonk(double)`
- **Geçersiz:** `int fonk(int)` ve `double fonk(int)`

Inline Fonksiyonlar

Normal Fonksiyon:

```
int kare(int x) {  
    return x * x;  
}  
  
int main() {  
    int a = kare(5);  
    // Fonksiyon cagrisi  
    // Zaman maliyeti var  
}
```

Inline Fonksiyon:

```
inline int kare(int x) {  
    return x * x;  
}  
  
int main() {  
    int a = kare(5);  
    // Kod gomulu: a = 5*5  
    // Cagri maliyeti yok  
}
```

Ne Zaman Kullanılır?

- Küçük fonksiyonlar (1-3 satır)
- Sık çağrılan fonksiyonlar

Dikkat: inline sadece bir öneridir, derleyici göz ardı edebilir.

Inline Fonksiyon - Normal Fonksiyon Karşılaştırması

Özellik	Normal	Inline
Çağrı maliyeti	Var	Yok
Kod boyutu	Küçük	Büyüyebilir
Hız	Daha yavaş	Daha hızlı
Kullanım	Büyük fonksiyonlar	Küçük fonksiyonlar

Örnek Kullanım:

- `inline int maks(int a, int b) { return (a > b) ? a : b; }`
- `inline double kare(double x) { return x * x; }`

Statik Değişkenler

Tanım

Fonksiyon içinde tanımlanan ancak değerleri fonksiyon çağrıları arasında **korunan** değişkenlerdir.

Normal Değişken:

```
void sayac() {  
    int s = 0;  
    s++;  
    cout << s << endl;  
}  
  
int main() {  
    sayac(); // 1  
    sayac(); // 1  
    sayac(); // 1  
}
```

Statik Değişken:

```
void sayac() {  
    static int s = 0;  
    s++;  
    cout << s << endl;  
}  
  
int main() {  
    sayac(); // 1  
    sayac(); // 2  
    sayac(); // 3  
}
```

Statik Değişken Örneği: Gol Atma

```
#include <iostream>
#include <string>
using namespace std;
void golAt(string oyuncu = "Bilinmeyen") {
    static int toplamGol = 0;
    toplamGol++;

    cout << oyuncu << " GOL ATTI!" << endl;
    cout << "Toplam gol: " << toplamGol << endl;
    cout << "-----" << endl;
}
int main() {
    golAt("Messi");      // Toplam: 1
    golAt("Ronaldo");    // Toplam: 2
    golAt("Neymar");     // Toplam: 3
    golAt("Messi");     // Toplam: 4
    golAt();             // Toplam: 5
    return 0;
}
```


Statik Değişken Örneği: Benzersiz ID Üretici

```
#include <iostream>
#include <string>
using namespace std;
int benzersizIDUret() {
    static int sonID = 1000;
    return sonID++;
}
void kullanıcıOlustur(string isim) {
    int id = benzersizIDUret();
    cout << "Kullanıcı: " << isim << " | ID: " << id << endl;
}

int main() {
    kullanıcıOlustur("Ali");           // ID: 1000
    kullanıcıOlustur("Ayse");          // ID: 1001
    kullanıcıOlustur("Mehmet");         // ID: 1002
    kullanıcıOlustur("Fatma");          // ID: 1003
    return 0;
}
```

Özyineleme (Recursion)

Tanım

Bir fonksiyonun kendisini çağırmasına özyineleme denir.

Faktöriyel:

```
int faktoriyel(int n) {
    if (n <= 1) {
        return 1; // Taban
    }
    return n *
        faktoriyel(n-1);
}

int main() {
    cout << faktoriyel(5);
    // 5! = 120
}
```

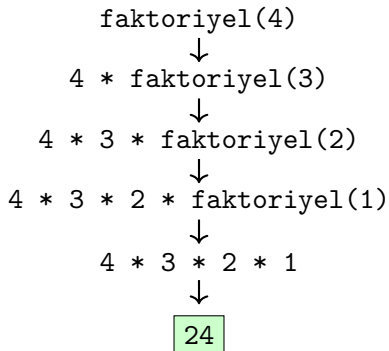
Fibonacci:

```
int fibonacci(int n) {
    if (n <= 1) {
        return n; // Taban
    }
    return fibonacci(n-1)
        + fibonacci(n-2);
}

int main() {
    cout << fibonacci(7);
    // 13
}
```

Özyineleme Nasıl Çalışır?

faktoriyel(4) çağırısı:



Uygulama 1: Alan Hesaplama

```
#include <iostream>
#include <cmath>
const double PI = 3.14159265359;
using namespace std;
double daireAlani(double r) {
    return PI * r * r;
}
double dikdortgenAlani(double uzunluk, double genislik) {
    return uzunluk * genislik;
}
double ucgenAlani(double taban, double yukseklik) {
    return (taban * yukseklik) / 2.0;
}
int main() {
    cout << "Daire (r=5): " << daireAlani(5) << endl;
    cout << "Dikdortgen (4x6): " << dikdortgenAlani(4,6) << endl;
    cout << "Ucgen (t=8,y=5): " << ucgenAlani(8,5) << endl;
    return 0;
}
```

Uygulama 2: Dizi İşlemleri

Fonksiyon Tanımları:

```
int diziToplam(int dizi[], int boyut) {
    int toplam = 0;
    for (int i = 0; i < boyut; i++) {
        toplam += dizi[i];
    }
    return toplam;
}

double diziOrtalama(int dizi[], int boyut) {
    return (double)diziToplam(dizi, boyut) / boyut;
}

int diziMaksimum(int dizi[], int boyut) {
    int maks = dizi[0];
    for (int i = 1; i < boyut; i++) {
        if (dizi[i] > maks)
            maks = dizi[i];
    }
    return maks;
}
```

Kullanım Örneği:

```
int main() {
    int sayilar[] = {12, 45, 23,
                     67, 34, 89};

    int boyut = 6;

    int toplam = diziToplam(sayilar, boyut);
    cout << "Toplam: " << toplam << endl;

    double ort = diziOrtalama(sayilar, boyut);
    cout << "Ortalama: " << ort << endl;

    int maks = diziMaksimum(sayilar, boyut);
    cout << "Maksimum: " << maks << endl;

    return 0;
}
```

Uygulama 3: Matematiksel Fonksiyonlar

Fonksiyon Tanımları:

```
bool asalMi(int sayi) {
    if (sayi <= 1) return false;
    if (sayi == 2) return true;
    if (sayi % 2 == 0) return false;

    for (int i = 3; i * i <= sayi; i += 2) {
        if (sayi % i == 0)
            return false;
    }
    return true;
}

int ebob(int a, int b) {
    while (b != 0) {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}
```

Kullanım Örneği:

```
int main() {
    // Asal sayi kontrolu
    int sayi = 17;
    if (asalMi(sayi)) {
        cout << sayi << " asaldır" << endl;
    } else {
        cout << sayi << " asal degildir" << endl;
    }

    // EBOB hesaplama
    int a = 48, b = 18;
    cout << "EBOB(" << a << ", " << b << ") = "
         << ebob(a, b) << endl;

    // EKOK hesaplama
    int ekok = (a * b) / ebob(a, b);
    cout << "EKOK(" << a << ", " << b << ") = "
         << ekok << endl;

    return 0;
}
```

Alıştırma Soruları - 1

- 1 İki sayının farkının mutlak değerini hesaplayan fonksiyon
- 2 Bir sayının çift mi tek mi olduğunu kontrol eden fonksiyon
- 3 Üç sayının ortalamasını döndüren fonksiyon
- 4 Faktöriyel hesaplayan fonksiyon (iteratif ve recursive)
- 5 Bir karakterin sesli harf olup olmadığını kontrol eden fonksiyon
- 6 Bir sayı dizisindeki en küçük elemanı bulan fonksiyon
- 7 Celsius'u Fahrenheit'a çeviren fonksiyon
- 8 Bir sayının basamak sayısını hesaplayan fonksiyon

Alıştırma Soruları - 2

- 1 Palindrom kontrolü yapan fonksiyon
- 2 Function overloading ile maksimum bulan fonksiyon (2 ve 3 sayı için)
- 3 Armstrong sayısı kontrolü
- 4 String içindeki kelimeleri sayan fonksiyon
- 5 Bubble sort ile dizi sıralama
- 6 N. Fibonacci sayısını hesaplayan fonksiyon (iteratif ve recursive)

Statik Değişken Soruları:

- Her çağrıda sayaç artıran fonksiyon
- Oyuncu başına gol sayacı
- ATM simülasyonu (paraYatir, paraCek, bakiyeGoster)

Alıştırma Soruları - 3

- 1 Binary search (özyinelemeli)
- 2 Hanoi Kuleleri problemi
- 3 String permütasyonları
- 4 Matris çarpımı
- 5 Sudoku geçerlilik kontrolü
- 6 Basit hesap makinesi (string değerlendirme)
- 7 Pascal üçgeni
- 8 Merge Sort algoritması
- 9 En yakın iki nokta bulma
- 10 Labirent çıkış yolu (backtracking)

Parametre Türleri:

- Değer (Pass by value)
- Referans (Pass by reference)
- Const referans
- Varsayılan parametreler

Özel Fonksiyonlar:

- Inline fonksiyonlar
- Overloading
- Özyinelemeli fonksiyonlar

Statik Değişkenler:

- Değer korunur
- Fonksiyon kapsamında
- Sayaç, ID üretici
- Global'den güvenli

İyi Pratikler:

- Açıklayıcı isimler
- Tek sorumluluk
- Az parametre
- Test edilebilirlik

Hatırlatmalar

- Fonksiyon isimleri açıklayıcı olmalı
- Her fonksiyon tek bir iş yapmalı
- Çok sayıda fazla parametre kullanmayın
- Büyük veri yapıları için const referans kullanın
- Özyinelemede mutlaka taban durum olmalı
- Inline sadece küçük fonksiyonlar için
- Overloading mantıklı olmalı
- Her fonksiyonu test edin

Soru&Cevap

Teşekkürler!